Progress Report: Dynamic Resolution for Task-Adaptive Efficient 3D Navigation and Target Detection

Mathieu Sauser (328777), Antonin HUDRY (341945), Ryan Gold (404941), Rokas Bertasius (407884)
CS-503 Project Progress Report {1,2}

I. Implementations

The environment logic is implemented in a custom `ThorEnv` class, the resolution degradation in a separate utility function, the visual and recurrent policy in `AgentPolicy` and `PolicyLSTM`, and the optimization loop in a `PPOAgent` class.

So far, we have implemented the full AI2-THOR environment and training pipeline in these steps:

Made resolution degradation function. It degrades original 224×224 resolution to the defined resolution level. Input image size is set to maximum LSTM resolution. For a given level, the image is divided into non-overlapping square blocks of size $2^{\text{level}} \times 2^{\text{level}}$. Each block is replaced by the average value of its pixels. The resulting low-resolution representation is then upsampled back to 224×224 using nearest-neighbor interpolation.

Implemented navigation following Gymnasium interface. At each step, the agent can do one action: move forward, backward, left, or right by 25 cm; rotate left or right by 45 degrees; look up or down by 15 degrees; use the sensing action (increases resolution by reducing its level by 1); or end the episode. When the agent moves/rotates/looks up or down, the observation is reset to the lowest resolution level.

Implemented reward system At the start of each episode, the environment resets the agent position, the sensing budget, and the target object. The agent then has at most 200 steps to find the target. An episode is considered successful when the target object is visible (1.5 m visibility distance) and close (< 0.5m from the agent). After each step, the environment computes a reward. The reward includes a positive terminal reward of $+5.0$ for success, a failure penalty of -1.0, a small step penalty of -0.002, a sensing penalty of -0.02, an additional oversensing penalty of -0.05 when the agent tries to sense without available budget or already at maximum resolution, and a collision penalty of -0.03 when an action fails. We also use distance-based reward shaping: whenever the agent gets closer to the target object, it receives a small positive reward proportional to the reduction in distance, with a scale factor of 0.01. These reward scales are mostly educated guesses and will be refined later on.

Implemented learning pipeline. For the learning pipeline, we implemented a recurrent actor-critic policy trained with Proximal Policy Optimization (PPO) [1]. At each timestep, the agent receives degraded RGB observation. The image is first passed through a frozen DINOv2 [2] pretrained Vision Transformer encoder which outputs a compact embedding of the current scene. Visual embedding is then concatenated with GPS position, compass orientation, previous action, current resolution level, and remaining sensing budget. The resulting vector is passed to an LSTM policy network, giving the agent short-term memory to integrate past observations and avoid revisiting the same locations. The LSTM output is collected over the episode and passed to two heads following an actor-critic architecture. The policy head (actor) outputs a distribution over discrete actions: navigation, sensing, and stop. The value head (critic) estimates the expected future return. Both are trained jointly using PPO, which updates the policy by maximizing a clipped surrogate objective to ensure stable learning, where clipping prevents excessively large policy updates:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right],$$

$$\text{where} \quad r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t, h_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t, h_t)}.$$

At timestep t, o_t is the agent's observation (visual embedding and auxiliary features), a_t is the selected action, and h_t is the LSTM hidden state. The ratio $r_t(\theta)$ measures how much more or less likely the updated policy is to select the same action compared to the policy that generated the trajectory. The advantage estimate $\hat{A}_t$ measures whether the action was better or worse than expected, estimated using generalized advantage estimation (GAE) [3]. In our environment, rewards include progress toward the target, sensing cost, collision penalties, step penalties, and terminal success or failure. In addition to the clipped policy objective, we use a value loss to train the critic and an entropy bonus to encourage exploration. The training loop records episode reward, episode length, success rate, number of sensing actions, final resolution level, and remaining sensing budget for later evaluation.

II. Comparison to the original proposal

Compared to the original proposal, the main architectural change is that the LSTM is not used directly as the visual encoder. Instead, we use a frozen pretrained Vision Transformer to extract visual features, and the LSTM is used as

the recurrent policy module. This design keeps the memory mechanism required for partial observability while providing stronger visual representations.

III. Discussion & Next Steps

At this stage, the codebase is established and mostly tested, and we are ready to begin experimentation. The main components of the environment and learning pipeline are in place, but training has not yet begun due to limited cluster access. Since this is our first RL project, part of the milestone was spent understanding how to design a stable training setup, define meaningful rewards, and connect the environment with the policy optimization loop.

One current limitation is reward design. The reward values were chosen based on intuition and common RL practice, but will likely require tuning once training begins. In particular, the sensing penalty must be carefully balanced. If too cheap, the agent may overuse high-resolution observations, if too expensive, it may avoid sensing entirely, making the adaptive perception mechanism useless. Reward tuning is expected to be one of the main tasks in the next experimental phase.

Another challenge is training time. AI2-THOR simulations are computationally expensive, and running a visual encoder at every timestep adds overhead. These costs are inherent to our setup, but parallelizing environment collection will increase trajectory throughput and stabilize PPO updates, reducing the total time required for training.

A remaining issue in the current setup is target conditioning. The environment defines a target object for each episode, but the target identity is not yet explicitly encoded in the policy input. Since the task is target-object navigation, the agent should know which object it is supposed to find. So, We plan to either add a target-object embedding to the auxiliary features or restrict the experiments to a fixed target.

The next major step is to implement the full evaluation pipeline. Our goal is to compare the adaptive-resolution agent against several baselines, all trained with PPO under the same setup, differing in how the sensing action is used:

- a low-resolution agent that never uses sensing;
- a high-resolution agent that always observes the scene at maximum resolution;
- a random-sensing agent that uses the sensing budget randomly;
- a fixed-schedule agent that senses at predefined intervals;
- our adaptive PPO agent, which learns when to use the sensing action.

We will evaluate all agents on task performance and perception efficiency. Task metrics include success rate, episode length, average return, and distance-to-goal reduction. If time allows, we will also compute Success weighted by Path Length (SPL), which is standard in navigation benchmarks. Perception efficiency metrics include average number of sensing actions, total sensing cost, remaining budget, and final resolution level. These metrics test whether the adaptive agent approaches the performance of the high-resolution baseline while using fewer high-resolution observations.

We also plan to analyze the learned sensing behavior, studying when the agent chooses to sense: early in the episode, after rotations, near target, before stopping, or when the budget is high. This is important to understand whether the agent learns a meaningful adaptive perception strategy.

Until the next progress report, our main action items are:

- finalize the PPO training loop and support parallel environment execution;
- add target-object conditioning or restrict the first experiments to a fixed target class;
- tune the reward parameters, especially the sensing penalty and terminal rewards;
- implement the low-resolution, high-resolution, random-sensing, and fixed-schedule baselines;
- build the evaluation pipeline and compute the selected metrics;
- run the first controlled training experiments on a small set of scenes;
- generate plots showing reward evolution, success rate, sensing frequency, and sensing budget usage.

These steps directly support the main goal of the project: determining whether an agent can learn to use visual resolution adaptively, improving the trade-off between navigation performance and perception cost.

IV. Author Contribution Statement

- Mathieu wrote the function that downgrade the image and `AgentPolicy`, wrote part of the training loop and most of the report.
- Rokas restructured and revised the report, organized the codebase, and contributed to early implementations.
- Antonin assembled most of the environment, structured the reward and made some visualization to check what the model observes.
- Ryan wrote the training loop and the training pipeline, helped with the environment information to model workflow, pipeline testing, edited the report and made the slides.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby *et al.*, "Dinov2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.